

4.1 参数高效微调简介

对于预训练数据涉及较少的垂直领域，大语言模型需要对这些领域及相应的下游任务进行适配。上下文学习和指令微调是进行下游任务适配的有效途径，但它们在效果或效率上存在缺陷。为弥补这些不足，参数高效微调（Parameter-Efficient Fine-Tuning, PEFT）技术应运而生。本节将首先对上下文学习和指令微调进行回顾，并分析它们的优缺点以及在实际应用中的局限性。接着，我们将详细介绍参数高效微调的概念及其重要性，阐述其在降低成本和提高效率方面的显著优势。随后，我们将对主流的 PEFT 方法进行分类，包括参数附加方法、参数选择方法和低秩适配方法，介绍每种方法的基本原理和代表性工作。

4.1.1 下游任务适配

通常，大语言模型通过在大规模数据集上进行预训练，能够积累丰富的世界知识，并获得处理多任务的能力 [43]。但由于开源大语言模型训练数据有限，因此仍存在知识边界，导致其在垂直领域（如医学、金融、法学等）上的知识不足，进而影响在垂直领域的性能表现。因此，需要进行下游任务适配才能进一步提高其在垂直和细分领域上的性能。主流的下游任务适配方法有两种：a) 上下文学习（In-context learning）[8]；b) 指令微调（Instruction Tuning）[53]。

1. 上下文学习

在之前的内容中，我们已经介绍过上下文学习的相关内容。它的核心思想是将不同类型的任务都转化为生成任务，通过设计 Prompt 来驱动大语言模型完成这些下游任务。小样本上下文学习（Few-shot in-context learning）将数据集中的样本-标签对转化为自然语言指令（Instruction）和样例（Demonstrations），并拼接上需要测试的样本一同输入给大语言模型，将模型输出作为最终预测结果。上下文学习完

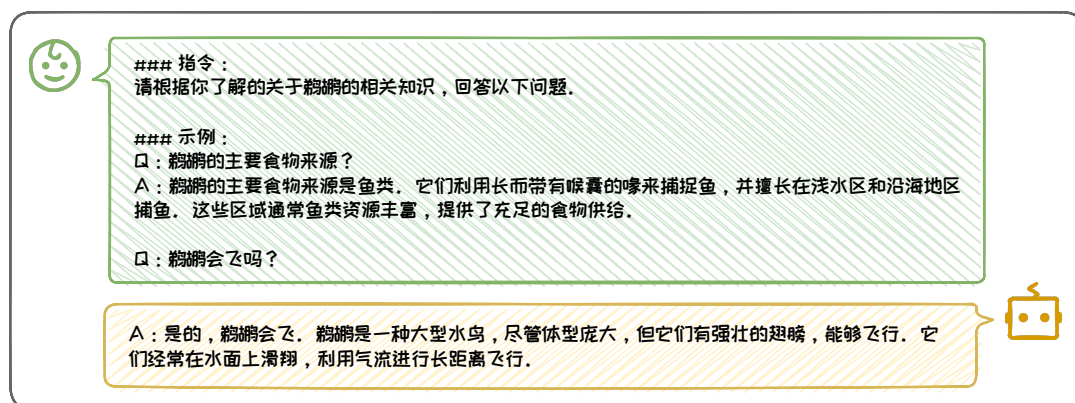


图 4.1: 指令数据样例。

全不需要对模型进行参数更新，因此能快速将单个模型应用到多种不同的任务上。

尽管在实际应用中，上下文学习能有效利用大语言模型的能力，但它缺点也很明显：1) 上下文学习的性能和微调依旧存在差距，并且 Prompt 设计需要花费大量的人力成本，不同 Prompt 的最终任务性能有较大差异；2) 上下文学习虽然完全不需要训练，但在推理阶段的代价会随 Prompt 中样例的增多快速增加。因此，微调大语言模型在许多场景和任务中依旧是必要的，尤其是在垂直领域。

2. 指令微调

指令微调 (Instruction Tuning) 是另一种进行下游任务适配的方法。指令微调旨在对模型进行任务指令的学习，使其能更好地理解 and 执行各种自然语言处理任务的指令。指令微调需首先构建指令数据集，然后在该数据集上进行监督微调。

- **指令数据构建**：指令数据通常包含指令（任务描述）、示例（可选）、问题和回答，如图 4.1 所示。构造这种指令数据一般有两种方式 [53]：1) 数据集成。通过使用模板将带标签的自然语言数据集，转换为指令格式的 < 输入，输出 > 对。如 Flan [47] 和 P3 [37] 数据集基于数据集成策略构建；2) 大语言模型生成。通过人工收集或者手写少量指令数据，再使用 GPT-3.5-Turbo 或 GPT4 等闭源大语言模型进行指令扩展。如 InstructWild [33] 和 Self-Instruct [46] 数据集采用这种方法生成。

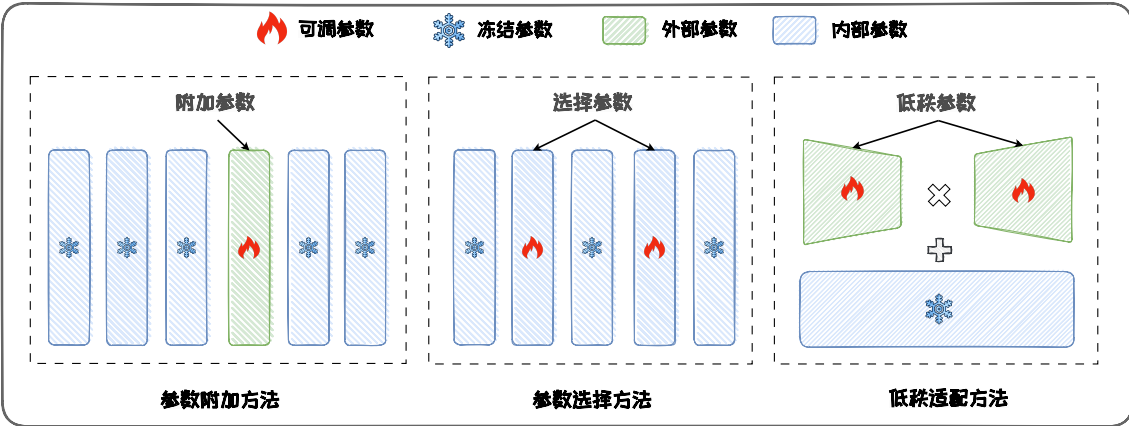


图 4.2: 高效参数微调方法分类学。

• **监督微调**：通过上述方法构建完数据集后，可以用完全监督的方式对预训练模型进行微调，在给定指令和输入的情况下，通过顺序预测输出中的每个 token 来训练模型。经过微调的大语言模型能够显著提升指令遵循 (Instruction-following) 能力，这有助于增强其推理水平，泛化到新任务和新领域。

尽管指令微调能有效帮助大语言模型理解新领域的知识，提高大语言模型在下游任务上的性能。然而，监督微调需要较大的计算资源，以 LLaMA2-7B [40] 模型为例，直接进行全量微调需要近 60GB 内存，普通的消费级 GPU（如 RTX4090 (24GB)）无法完成微调。因此，为了在资源受限的环境中有效微调大语言模型，研究参数高效的微调技术显得尤为重要。

4.1.2 参数高效微调

参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT) 旨在避免微调全部参数，减少在微调过程中需要更新的参数数量和计算开销，从而提高微调大语言模型的效率。主流的 PEFT 方法可以分为三类：参数附加方法 (Additional Parameters Methods)，参数选择方法 (Parameter Selection Methods) 以及低秩适配方法 (Low-Rank Adaptation Methods)，其方法思想如图 4.2 所示。

1. 参数附加方法

参数附加方法 (Additional Parameters Methods) 在模型结构中附加新的、较小的可训练模块。在进行微调时，将原始模型参数冻结，仅微调这些新加入的模块，从而来实现高效微调。这些模块通常称为适应层 (Adapter Layer)。它们被插入到模型的不同层之间，用于捕获特定任务的信息。由于这些新增的适应层参数量很小，所以参数附加方法能够显著减少需要更新的参数量。典型方法包括：**适配器微调 (Adapter-tuning)** [18]、**提示微调 (Prompt-tuning)** [23]、**前缀微调 (Prefix-tuning)** [24] 和 **代理微调 (Proxy-tuning)** [27] 等。参数附加方法将在 4.2 节进行具体介绍。

2. 参数选择方法

参数选择方法 (Parameter Selection Methods) 仅选择模型的一部分参数进行微调，而冻结其余参数。这种方法利用了模型中仅有部分参数对下游任务具有决定性作用的特性，“抓住主要矛盾”，仅微调这些关键参数。选择性地微调这些关键参数，可以在降低计算负担的同时提升模型的性能。典型的方法包括：**BitFit** [50]、**Child-tuning** [49] 以及 **FishMask** [39] 等。参数选择方法的将在 4.3 节具体介绍。

3. 低秩适配方法

低秩适配方法 (Low-rank Adaptation Methods) 通过低秩矩阵来近似原始权重更新矩阵，并冻结原始参数矩阵，仅微调低秩更新矩阵。由于低秩更新矩阵的参数数量远小于原始的参数更新矩阵，因此大幅节省了微调时的内存开销。LoRA [19] 是经典的低秩适配方法，后续有 **AdaLoRA** [52]、**DyLoRA** [42] 以及 **DoRA** [29] 等变体被提出，进一步改进了 LoRA 性能。低秩适配方法将在 4.4 节具体介绍。

4.1.3 参数高效微调的优势

参数高效微调有以下优势：1) **计算效率高**：PEFT 技术减少了需要更新的参数数量，从而降低了训练时的计算资源消耗；2) **存储效率高**：通过减少需要微调的参